

DEGREE PROJECT FOR MASTER OF SCIENCE WITH SPECIALIZATION IN ROBOTICS
DEPARTMENT OF ENGINEERING SCIENCE
UNIVERSITY WEST

Enhancing Efficiency in Metal 3D Printing Through Heat Simulation and AI

Jean Justine



A THESIS SUBMITTED TO THE DEPARTMENT OF ENGINEERING SCIENCE
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE WITH SPECIALIZATION IN ROBOTICS
AT UNIVERSITY WEST
2025

Date:	14/05/2024
Author:	Jean Justine
Examiner:	Fredrik Sikström
Advisor:	Xiaoxiao Zhang, Högskolan Väst
Programme:	Master in AI and Automation
Main field of study:	AI with a specialization in Automation
Credits:	21 Higher Education credits
Keywords:	Automation, 3D printing, Simulation, Reinforcement Learning, AI
Template:	University West, IV-Master 2.10
Publisher:	University West, Department of Engineering Science S-461 86 Trollhättan, SWEDEN Phone: + 46 520 22 30 00 Fax: + 46 520 22 32 99 Web: www.hv.se

Abstract

Printing complex and unique form have an increasing demand in industry, answering this, metal 3D printing is a good alternative. It offers a possibly to print pieces without having to create a new production chain and facilitates the access to them. However, as a growing demand and a relatively new method, there is a room for improvements. This thesis aims to answer this request, researching how to improve the efficiency through heat simulation and AI. Before presenting the results, here are the major steps to achieve that. Firstly, creating a heat simulation using a voxel representation and a heat equation to fit each piece geometry. Using Robot studio to simulate the printing, everything is set up to train an AI except data. To counter this problem, the use of reinforcement learning, which doesn't require much data to be trained, will be the model of AI.

The results of this thesis work are an improvement in time saving compared to a simple algorithm, at best 13%-time savings, as well as an accurate representation of pieces and an acceptable thermal simulation. Every work done during this thesis is available on GitHub [1].

Affirmation

This master degree report, Enhancing Efficiency in Metal 3D Printing Through Heat Simulation and AI, was written as part of the master degree work needed to obtain a Master of Science with specialization in Robotics degree at University West. All material in this report that is not my own is clearly identified and used in an appropriate and correct way. The main part of the work included in this degree project has not previously been published or used for obtaining another degree.



Signature by the author

Jean Justine

14/06/2025

Date

Contents

ENHANCING EFFICIENCY IN METAL 3D PRINTING THROUGH HEAT SIMULATION AND AI	I
ABSTRACT	III
AFFIRMATION	IV
CONTENTS	V
SYMBOLS AND GLOSSARY	VII
1 INTRODUCTION	1
1.1 AIM.....	2
2 RELATED WORK (BACKGROUND)	3
2.1 WELDING PROCESS FUNDAMENTALS.....	3
2.2 THERMAL BEHAVIOR IN WAAM	3
2.3 ROBOTIC SIMULATION IN WAAM.....	4
2.4 VOXEL-BASED GEOMETRY FOR 3D PRINTING REPRODUCTION	4
2.5 VOXEL-BASED GEOMETRIC DISCRETIZATION FOR THERMAL MAPPING	4
2.6 REINFORCEMENT LEARNING IN ADDITIVE MANUFACTURING	4
3 METHOD	6
3.1 ROBOT STUDIO SIMULATION	6
3.2 FETCHING THE COORDINATE	6
3.3 VOXEL REPRESENTATION	6
3.3.1 Mapping deposition points to Voxel Indices.....	6
3.3.2 Local 2D Filling of Each Deposition Point.....	7
3.3.3 Vertical Smoothing Across Layers.....	7
3.3.4 Extraction of Per-Layer Bounding Boxes.....	7
3.4 COOLING TIME.....	7
3.5 HEAT SIMULATION.....	7
3.5.1 Data Loading and Initialization.....	7
3.5.2 Heat-equation Formulation	8
3.5.3 Geometry Analysis	8
3.5.4 In-Slice Neighbourhood Averaging	8
3.5.5 Layer-Wise Time Integration	8
3.5.6 Saving heat information	8
3.6 CHOOSING PATH ALGORITHM.....	8
3.7 REINFORCEMENT LEARNING IMPLEMENTATION	9
3.7.1 Justification of Reinforcement Learning	9
3.7.2 Problem Formulation	9
3.7.3 Q-Learning Algorithm	9
3.7.4 Integration into the Printing Loop	9
3.8 EXPERIMENT	10
3.8.1 Objectives	11
3.8.2 Experimental Setup.....	11
3.8.3 Protocol.....	11
3.8.4 Results Summary	11

3.9	SOURCES OF ERRORS AND ERROR MARGINS	11
4	WORK	12
4.1	VIRTUAL WAAM ENVIRONMENT SETUP	12
4.2	TCP DATA ACQUISITION.....	13
4.2.1	<i>Outlier Detection & JSON Logging Structure</i>	<i>13</i>
4.3	VOXEL-BASED GEOMETRY PIPELINE	14
4.3.1	<i>Voxel Grid Parameters (resolution, layer thickness).....</i>	<i>14</i>
4.3.2	<i>Deposition-Point Voxel-Index Mapping</i>	<i>14</i>
4.3.3	<i>In-Plane Filling & Vertical Smoothing</i>	<i>14</i>
4.3.4	<i>Bounding-Box Extraction and Storage</i>	<i>14</i>
4.4	TRANSIENT HEAT-TRANSFER SIMULATION.....	15
4.4.1	<i>ODE Solver Implementation for Each Voxel.....</i>	<i>15</i>
4.4.2	<i>Empirical Parameter Calibration (α, β, γ)</i>	<i>15</i>
4.4.3	<i>In-Slice Neighbourhood Averaging & Time-Stepping.....</i>	<i>15</i>
4.4.4	<i>Saving the heat map.....</i>	<i>15</i>
4.5	REINFORCEMENT-LEARNING DRIVEN BUILD-SEQUENCE SCHEDULING	16
4.5.1	<i>Design of the Reward Function.....</i>	<i>16</i>
4.5.2	<i>Scheduling Algorithm Logic</i>	<i>16</i>
4.5.3	<i>Q-Learning Algorithm and Hyperparameters</i>	<i>17</i>
4.5.4	<i>Integration with Thermal Model and Python Pipeline</i>	<i>17</i>
4.6	EXPERIMENTAL TRIALS AND VALIDATION.....	17
4.6.1	<i>Selection of Test Geometries and Simulation Parameters.....</i>	<i>17</i>
4.6.2	<i>Trial Protocol (number of runs, data captured)</i>	<i>20</i>
4.6.3	<i>Performance Metrics and build time saving.....</i>	<i>21</i>
4.6.4	<i>Error Sources and Uncertainty Quantification</i>	<i>21</i>
5	RESULTS AND DISCUSSION	22
5.1	SUMMARY OF KEY FINDINGS	22
5.2	DETAILED ANALYSIS OF EXPERIMENTAL RESULTS	22
5.3	REINFORCEMENT LEARNING MODEL PERFORMANCE	23
6	CONCLUSION	24
6.1	FUTURE WORK AND RESEARCH.....	24
6.2	CRITICAL DISCUSSION.....	25
6.3	GENERALIZATION OF THE RESULT	25
7	REFERENCES.....	26

Symbols and glossary

DED	Direct Energy Deposition, a method of metal 3D printing it uses a laser to melt a metal wire into the shape requested.
WAAM	Wire Arc Additive Manufacturing, it is a method of metal 3D printing that is used during this thesis, it works with a robot arm that has a metal wire and is welds using an arc onto pieces that are being printed. It is a branch of DED.
TCP	Tool Center Point represents the tip of the tool where it is used in Robot Studio.
ODE	Ordinary Differential Equation, this is an equation relating a function to its derivatives.
AI	Artificial Intelligence.
ML	Machine learning is a branch of AI in which models are created and trained on data.
RL	Reinforcement Learning, a Machine Learning branch in which the model learns with a system of reward and penalty to optimize what it is trained for.

1 Introduction

Metal additive manufacturing has emerged as a transformative technology in multiple domains, such as medical, aerospace, and others. It offers the ability to produce complex and unique parts that could be hard or impossible to produce with conventional methods and machines. Those demands continue to rise as companies seek better performance and easier access.

It is in this context that metal 3D printing takes place, allowing manufacturers to create complex parts more easily and quickly without having to create a new production chain. However, metal 3D printing faces multiple challenges, such as inefficient build sequences, a lack of heat propagation management. Moreover, each new geometry requires a new approach, new parameter tuning, resulting in trial-and-error cycles, using both time and hardware.

In metal additive manufacturing, as well as metal 3D printing, Thermal Management is crucial. The intense, localized heating energy from the source creates a very varied thermal gradient within the piece. If these gradients are not controlled, the piece can be ruined, distorted, not smooth, etc. Solving this problem is key, to do so, simulating the heat and printing of the pieces, as accurately as possible, is mandatory, especially for pieces that are more complex than others and will have different temperatures and cooling times between printing.

The metal 3D printing machine is simulated using Robot Studio in a virtual environment designed after the real model. The simulation brings a completely flexible environment to answer the different needs. It also brings an infinite amount of experimentation, whereas the real model costs money and time to operate.

The different pieces printed are represented using voxels to allow easy access to those pieces' geometry, form, and to do the heat simulation on each of them. The voxel representation is crucial for this process. The heat can be calculated and represented on it and then used to train an AI model, or to make decisions about the printing schedule regarding the piece's heat condition.

Other similar experiments were conducted beside this one, such as improving the direct printing of different pieces using AI to optimize the path of the arm printing one layer, but a few were conducted to improve the order of the pieces to print as well as a heat simulation that allows decisions to be made; the work done during this thesis is innovating with the use of AI model, as well as the simulation of the heat.

This method allows a solid base to improve the efficiency of the metal 3D printing machine as well as the use of other methods if needed, it is versatile regarding the possibility of different experiments that may be conducted on it.

AI is a way to reduce waiting time, and can learn quickly to enhance the printing path schedule and which pieces to print first. By doing so, it can reduce the total time to print pieces, maximize the quality, and minimize the time spent. Also the heat simulation prevents any form of malformation due to too high temperatures and bad scheduling of the path, resulting in printing another layer when the piece is not ready yet and has not cooled enough to be back to normal temperature.

1.1 Aim

To address these challenges, this thesis aims to develop a comprehensive simulation environment that supports AI-driven optimization of metal 3D printing and a complete simulation of the heat of pieces that are printed, as well as a representation of their pattern and geometry features.

The objectives are:

- Develop and validate a physics-based model to reproduce the real metal printing process and ensure its realism.
- Create an accurate representation of different pieces printed.
- Simulate the thermal environment of those printed pieces.
- Implement and evaluate algorithms, such as reinforcement learning, to autonomously optimize the build sequence.

2 Related work (Background)

In metal 3D printing, there are different methods, such as Laser Powder Bed Fusion, that cover multiple sub-methods like selective laser printing, which deposit a layer of powder and use a laser to imprint on those layers the pieces they want [1].

The method that is worked on during this thesis is a branch of the one called Direct Energy Deposition (DED), which uses a metal wire or powder and melts it using an energy source such as a laser [2].

During this thesis, the method to print is called WAAM, Wire Arc Additive Manufacturing, it consists of a wire going through an arm robot from ABB studio, and an electrical arc that melts the wire directly onto the plate, then builds layer by layer the piece requested [3].

All those methods differ in terms of capability, energy, build rates, and the possibility of geometry to print.

The one used, WAAM, has multiple advantages compared to other ways of printing; it can print large pieces, as it is only limited by the size of the robot. It uses common welding wires that are easy to find, already used for a long time, so it is rather optimized. Also, it can print a lot of material quicker than others DED [3].

2.1 Welding Process Fundamentals

Arc-based WAAM, the process derives directly from gas-metal arc welding, combining with additive manufacturing, it is produced by a robot that deposits a layer of metal on top of each other until the pieces wanted are ready.

To simulate the moving heat source, WAAM models mostly used: double ellipsoid Goldak equivalent heat source. In a coordinate system moving with a torch or arc welding, the front and rear volumetric power densities are given as:

$$\begin{aligned}
 q_f(x, y, z, t) &= (6\sqrt{3}ffQ/afbc\pi) \exp(-3(x+v(\tau-t))^2/af^2) \exp(-3y^2/b^2) \exp(-3z^2/c^2) \\
 q_r(x, y, z, t) &= (6\sqrt{3}frQ/arbc\pi) \exp(-3(x+v(\tau-t))^2/ar^2) \exp(-3y^2/b^2) \exp(-3z^2/c^2) \\
 ff + fr &= 2 \\
 2Q &= (q_f(x, y, z, t) + q_r(x, y, z, t)) dx dy dz \\
 Q &= \text{Heat, [W]}
 \end{aligned}$$

Where Q is an arc power, v the torch speed, (af/r, b, c) the ellipsoid semi-axes, and ff/fr for the front/rear energy fractions [4].

2.2 Thermal Behavior in WAAM

Using WAAM involves a thermal reaction; it is governed by the transient heat conduction equation with moving sources:

$$\rho C_p \partial T / \partial t = \nabla \cdot (k \nabla T) + q(x, y, z, t),$$

where ρ is the density, C_p the heat capacity, k the thermal conductivity, and q the spatially distributed Goldak source term [5].

However, during this thesis, the heat equation that is adopted is a simpler format, a lump parameter thermal model to track the temperature per layer of the form,

$$dT/dt = \alpha(n)Q(t) - \beta(n)[T_1(t) - T_{amb}] - \gamma(n)[T^4(t) - T_{amb}^4].$$

Where:

- $T_1(t)$ is the mean temperature of the deposited material at time t ,
- $Q(t)$ is the instantaneous arc power input (voltage $\times$ current),
- T_{amb} is the ambient temperature,
- $\alpha_1(n_1)$ maps power input to temperature rises and may vary with layer index n_1 ,
- $\beta_1(n_1)$ is the coefficient for linear heat losses (convection/radiation),
- $\gamma_1(n_1)$ captures nonlinear radiative losses.

2.3 Robotic Simulation in WAAM

High-fidelity reproduction of the WAAM kinematics has been achieved through offline programming in ABB RobotStudio. Tools and pieces are as real as possible to achieve a perfect model that allows reproduction of the 3D metal printing machine as close to reality as possible. To work in an environment that saves time, resources, allows errors, and a nearly full control over it [6].

2.4 Voxel-Based Geometry for 3D printing reproduction

3D printing reproduction under a voxel-based geometry is a method that allows the recreation of any shape desired. In 3D printing, it is used to have high quality, with full resolution, and allows the reproduction of shapes, achieving high fidelity. This allows numerous forms of application, notably to simulate the heat, or to have the same quality as the pieces printed [7] and [8].

2.5 Voxel-Based Geometric Discretization for Thermal Mapping

Thermal mapping allows a lot of computation to simulate the heat correctly. When a coordinate is imported, it is first discretised into a uniform 3D structure of cubic voxels. Each of those voxels is unique and computed. This allows to have a full rendering of different pieces in 2D or 3D. With that, calculating every different element or part becomes doable. Thermal mapping is then realized by implementing different heat equations required, and the output is the final pieces or objects with a full heat map [9].

2.6 Reinforcement Learning in Additive Manufacturing

Reinforcement Learning (RL) has emerged those recent years to become a powerful tool for sequential decision-making problems in which the optimization of a set of actions to maximize long-term reward is required. RL enables adaptive, closed-loop control, time-varying process.

In layer-by-layer methods such as DED or WAAM, key parameters such as print speed, extrusion temperature, piece temperature, layer height... strongly influence quality. RL models are mostly trained on simulation to change and optimize those key parameters to achieve good results without having to go through mathematical models [11].

RL has also been used to optimize path planning for different pieces. Choosing a path where the high-dimensional space of trajectories is complex, it is often found using trials and errors, all in a simulation. Models learn how to generate paths that minimize errors, movement, layer misalignment, as well as thermal distortion. Also, in some work, where the RL is used on the real machine and takes the decision, thermal camera helps find errors, and the model corrects them in real time [11] and [11], [13], [13].

3 Method

For this thesis, multiple methods were used to complete it. Going from simulation on ABB Robot Studio to Voxel Representation, to finally create a heat simulation that sticks to reality. All the following steps are in chronological order; some previous steps were modified later on if something needed to be changed or did not work with the previous one.

3.1 Robot Studio Simulation

Robot Studio (v2024) was selected because it provides an industry-standard digital twin of the ABB manipulator and a well-documented Python API. Also, it allows a multitude of tools as well as complete control over the environment, like the speed, the pieces to print, the code, and variables.

To achieve that, Robot Studio was used to simulate the metal 3D printing machine using the same speed and paths that represent the pieces. Multiple variables are created to allow easy access to the coordinate of the tool center point (TCP), whether the machine is Welding or not, the number of layers per piece, which pieces are being printed, and another one to allow the machine to print or not.

3.2 Fetching the coordinate

From the code in Python, multiple functions are created that access those variables or set a variable. For instance, a function called “fetch_xyz” allows the importation of the coordinate. On top of that, there is also a verification that a piece is being printed and that the arm is being welded. If it is finished, fetching is stopped, and a variable is set that stops the Rapid code in Robot Studio until it is allowed to run again. During all this fetching time, a json file is appended with every coordinate previously collected.

When the fetching time is done and the json is available, all the small outliers that could have been fetched by mistake are sorted out.

3.3 Voxel Representation

The following steps convert raw TCP logs into per-piece voxel models. It consists of multiple steps to allow a very fitting representation of the different pieces that are fetched. This method was chosen over mesh or point-cloud formats due to its advantages, such as saving space, its accuracy, it is also widely used, allowing the representation of 3D geometrical form and a direct compatibility with the heat equation that will be used.

3.3.1 Mapping deposition points to Voxel Indices

Firstly, the TCP brute coordinate (‘x’, ‘y’, ‘z’) is converted into integer voxel indices (‘ix’, ‘iy’, ‘iz’) by first normalizing each axis to a 0,1 range based on the observed minimum and maximum. So, they correspond to the first grid position and the last. This places each point proportionally within the grid. Then those values are rounded to the nearest integer. The same principle is applied for each vertical coordinate, but if a fixed layer thickness is specified, each vertical coordinate is divided by this thickness to assign it to a particular layer index

3.3.2 Local 2D Filling of Each Deposition Point

Each mapped voxel (ix, iy, iz) is expanded into a small square footprint in its Z slice to represent the toolhead diameters and the surface printed. A radius defines a neighbourhood of offsets (dx,dy), for each offset within the radius, the corresponding voxel grid cell at (ix+dx, iy+dy) is filled. It is to ensure that small gaps between sampled points are connected and closed within the same layer.

3.3.3 Vertical Smoothing Across Layers

To reduce layer-to-layer discretization noise, a sliding window of adjacent Z slices, with a default window of 3, is checked for each slice index z. The sum of binary values across the window is compared to half the window depth. If the local sum meets or exceeds that threshold, the voxel at (x,y,z) is also filled. This majority voting system ensures the vertical consistency across all layers. It is not used on low-resolution voxels, but if the resolution goes higher, its use is mandatory.

3.3.4 Extraction of Per-Layer Bounding Boxes

Connected regions in the binary voxel grid are first identified using a 3x3x3 structure to assign each cluster a label. For each label, all voxel coordinates are grouped by their Z-slice, and the minimum and maximum X, Y indices in each slice define that layer's 2D bounding box.

Then, a nested dictionary mapping each component label to its per-slice "bounding_box" and list of "active_pixels", and then saved it as json.gzipped file. This compact layer representation allows for savings of space without a drop in the quality of the representation.

3.4 Cooling time

Cooling intervals were computed for each piece by comparing timestamps at the end of each layer. An internal dictionary that maps each piece to its cool time was created and enables on-demand retrieval of elapsed cooling time.

When printing of a layer begins, the current time is fetched and compared to the stored end time for that piece. The difference defines the cooling time since the last build of the piece; this value is returned, and the last end timestamp is reset.

If a piece's layer is complete, the end function records the current timestamps in the dictionary, and these timestamps become the reference for all future cooling time calculations for that piece.

All timing is based on Python's perf counter, allowing a very high precision.

3.5 Heat simulation

The voxelized geometry done previously can now be subjected to a transient heat transfer simulation that solves ordinary differential equations for each active voxel over successive layers. This procedure is implemented in multiple steps

3.5.1 Data Loading and Initialization

The gzipped json file of each representation produced in section 3.3 is decompressed and parsed to reconstruct the pieces per slice voxels map. A three-dimensional temperature grid of size (nz * nx * ny) is initialized to the ambient temperature for every voxel.

3.5.2 Heat-equation Formulation

A local Heat ordinary differential equation (ODE) is defined for each active voxel:

$$dT/dt = \alpha Q - \beta[T - T_{amb}] - \gamma[T^4 - T_{amb}^4].$$

Where:

- Q is the constant heat input per step.
- α , β , γ are empirical parameters that depend on the local geometry of the pieces as well as their cooling time and the number of layers.
- T is the current voxel temperature.
- T_{amb} is the ambient temperature.

3.5.3 Geometry Analysis

A geometry analysis of the different pieces is conducted to obtain the compactness, filled area, maximum internal gap, number of walls, the average thickness of the wall, as well as the bounding box area.

3.5.4 In-Slice Neighbourhood Averaging

To approximate lateral heat conduction (among the layers), each temperature is blended with the mean temperature of its eight adjacent neighbours on the same Z slice. A weighted average is then used as the temperature in the ODE.

3.5.5 Layer-Wise Time Integration

The simulation advances layer by layer, for each of them, a specified number of sub-steps is computed, with a time increment dt , in addition to the cooling time that influences the ODE equation. During each substep, all active voxels in the current layer are updated at once by adding $dt \times ODE$ to their temperature. The updated grid with new temperatures serves then for the initial condition for the next layer.

3.5.6 Saving heat information

To save the computed voxel with the new temperature, the heat map is saved under an .npy file and a new internal dictionary called stats is created that stores, the average temperature of the piece's active voxel, without taking in account the "air", its dimension as well as its cool time and the name of the files where the heat map are saved.

3.6 Choosing Path Algorithm

To determine which pieces to print first, a simple algorithm is created.

Once all the pieces had one layer printed. A list of the piece's id is created, and an algorithm go through the stats dictionary previously created and selected the pieces with the lowest average temperature to be printed, if this temperature is below the threshold where it is allowing to print on, it print it and its number of layer is incremented by one, if it's not allowed, it's waiting 10 seconds to add to the cool time, it compute again the heat map as well as the average temperature of all pieces and it start again. Once a piece has reached the number of layers it needs, this piece's ID is removed from the list, and only the other pieces can now be printed.

3.7 Reinforcement Learning Implementation

This section describes how a Q-learning agent was integrated into the pipeline, which makes decisions on which pieces to print based on its temperature, cooling time, and number of layers.

3.7.1 Justification of Reinforcement Learning

A simple model of RL was chosen because the thermal dynamics, time constraints of each part are complex time time-varying, and not easily understood by a normal algorithm. Q-learning allows to learn an optimal scheduling policy directly from the simulation and is easy enough to work for selecting which pieces to print next.

3.7.2 Problem Formulation

- State (S): at each decision point, the state is encoded as a tuple per active piece (avg_temp/10, cool_time./10, layer_count) where:
 - Cool time is the time since the last time the piece has been printed.
 - Avg temp is the average temperature of these pieces
 - Layer count is the number of layers printed so far
- Action (A): Selecting one piece from the set of pieces where their average temperature is below the threshold.
- Rewards (R):
 - Always 1 as a baseline penalty
 - +5 if a piece's layer is printed immediately (no waiting time)
 - +1 if there is less than 30 seconds of waiting
 - -5 in other cases

3.7.3 Q-Learning Algorithm

- Update rule:
 - $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \cdot a' \max_{a'} Q(s', a') - Q(s, a)]$
 - Q is the current estimate value of the action a in the state s , $a \in [0, 1]$ the learning rate, r the reward, $\gamma \in [0, 1]$ the discount factor, s' the following state after taking action a , $\max_{a'} Q(s', a')$ is the estimated best possible value for the next state s' over all possible actions a .
- **Policy:** An ϵ -greedy strategy was implemented, with initial $\epsilon=0.2$ decayed by 0.995 per episode to a minimum of 0.05.

All Q-table operations and policy decisions were handled by the class Q Agent in the q_agent.py file.

3.7.4 Integration into the Printing Loop

To integrate the model into the printing loop, for it to make a decision on which pieces to print, it is first loaded, and if the model wasn't saved or was deleted, a blank model is created called Q-table.

The workflow of the RL model is as follows:

- The cooling time for each piece is retrieved with the "get_cooling_time" function.
- Thermal information is updated with "save_heat_stats," which saves the statistics and calls the main heat function, and is accessed via this dictionary.

- The number of layers already printed for these pieces is retrieved with the “fetch_number_of_layers” function that connects to Robot Studio and takes the value directly.
- The current set is encoded via “agent.encode_state(stats, piece_id)”.
- A valid action set was filtered by temperature threshold.
- The model chose an action using “agent.choice_action”, among the available pieces that can be printed, then it prints the corresponding choice using the “set_piece_choice()” function.
- After execution, a reward was computed, and “agent.update(...)” was used to adjust the Q-table.
- ϵ , the greedy strategy was decayed via agent.decay_epsilon().

This loop continued until all parts reached the target layer count, at which point the Q-table was saved.

As seen in Figure 1, it depicts how the RL model is integrated in the rest of the pipeline. After, Robot Studio prints one layer of every piece, the Python code creates the Voxel representation of each layer, saves it under a json.gz file, calculates its heat map and saves it under npy, calculates also its average temperature, and already has its cooling time.

The cooling time, average temperature, and how many layers the piece has is given to the RL model as parameters, and then take as an action which piece to print. It is sent through Python using Robot Studio API, and then prints it. The RL model then receives a reward according to the reward system to minimize the time to wait in between pieces.

Then the loop starts again with the representation of the voxel and then the heat simulation. It loops until all the layers are printed out.

3.8 Experiment

To make every bit of code work, from robot studio to python, to do a voxel representation that works, connect python to robot studio, create the heat simulation and check if the results are realists, find out if the choosing path algorithms work and save time, implement a RL model, took approximately five hundred attempt that were made during this thesis.

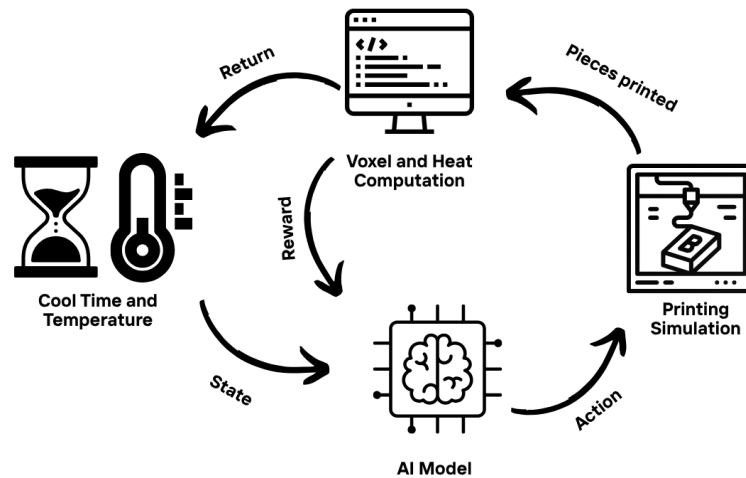


Figure 1 Reinforcement Learning Architecture implemented in the pipeline

3.8.1 Objectives

- Verify that the pipeline Robot Studio, Voxel, Heat Simulation produces temperature within a realistic range.
- Evaluate the efficiency of the path-selection algorithm in reducing total build time.
- Evaluate the efficiency of the RL model in reducing total build time.

3.8.2 Experimental Setup

The equipment used in this thesis is a laptop, Lenovo Legion, with 24 gigabytes of RAM, an Intel Core i5 processor, as well as 6 GB of Video RAM using Nvidia RTX 2060.

The simulation of the printing machine was done through Robot Studio.

The Python pipeline was used with Visual Studio Code with Python v3.9.1.

3.8.3 Protocol

The experiments were carried out step by step:

For each geometry and run:

- Simulate one layer in Robot Studio while recording TCP coordinates.
- Generate voxel model and compute heat profile for all layers of every piece.
- Record the Average temperature and cooling time.
- Apply the path selection algorithm and the RL model to find which pieces to print.
- Record the total simulated build time for the two methods and compare it to a simple path algorithm.

3.8.4 Results Summary

In total, approximately 500 runs were affected, with 250 to create the fetching of coordinates, as well as the voxel representation, then 150 to create the heat simulation as realistically as possible, and finally 100 to test the path choosing algorithm, as well as the RL model.

3.9 Sources of errors and error margins

Simulation vs reality: as no data on the actual machine was available, a consequent margin on the thermal simulation was allowed.

Voxel resolution may neglect features under 2 mm due to the reduction and allowed computation of the size of the voxel, it can be reduced even further with a higher voxel resolution.

The cooling time measurement error is negligible.

The sources of errors come mainly from the code's errors, as well as all the calculations that needed to be changed to fit reality, resulting in a lot of experimentation.

4 Work

The work conducted during this thesis has multiple steps, and every one of them has to be connected between each other's perfectly, otherwise errors or miscalculations would have appeared, which is why it requires precise and flawless execution.

4.1 Virtual WAAM Environment Setup

A virtual simulation of the machine 3D printing was created in Robot Studio that mimics the movement and speed of the real arm, also multiple paths for different pieces were created. The movement when printing is 12 mm/s, and in between printed during transition phases, it's 800 mm/s. The TCP, world base coordinate, and each piece coordinate were defined as separate tools and work objects. When a piece is printed, it sends a signal call "weld" to indicate when it's welding. This signal turns to false when it is in transition phases, as well as when the arm is reorienting itself in the same piece phase. At the end of every layer's print, the work object z dimension of this piece is incremented by 1 to simulate the layer's thickness, whereas the tool work object doesn't change, so the TCP can print above the layer. When all the printing is finished, the piece's work object is put back to normal at $z = -100$. Also, when a layer is done, the program waits for an order from Python's main code to go

and select a new piece to print.

Every piece will be printed with a defined number of layers, to train and optimize everything, the number of layers printed throughout the whole thesis was from 2 to 15. As seen in Fig. 2, the raw path of the machine to print one layer of piece number one. This part aligns with the aim of creating a realistic simulation by creating a twin of the real machine and real pieces.

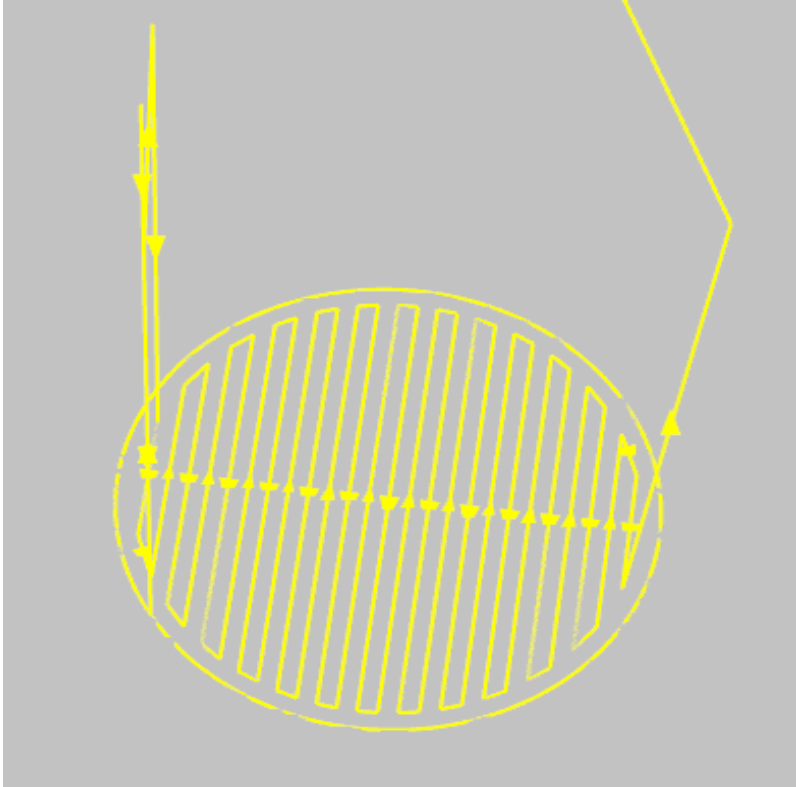


Figure 2 Raw path of the printing of piece 1 in Robot Studio

4.2 TCP Data Acquisition

To acquire the TCP coordinate, a function called “fetch_xyz” is implemented in another function that fetch continuously: the TCP coordinate with “fetch_xyz”, if it’s welding or not with “fetch_weld”, and the state of the layer, if it’s also finished or not. The coordinate are fetch at the frequency of 1kHz as the condition that the simulation is welding and the layer is not finished, those coordinate are save in a json file called “depositions_points_piece_{piece_id}.json”, so there is a separate json for every pieces. Depending on the piece’s geometry, but approximately 25 points are fetch each seconds when printing and fetching. Those are reset every time the main function is called. In Fig 3, the representation of all the points for one layer fetched are put into 3D.

4.2.1 Outlier Detection & JSON Logging Structure

Some outliers could have slipped by even with the welding and layer’s security, to fix this problem, the json of the current piece is load in “deposition_point” variable, then the outliers are filter out by the function “filter_points_by_layer“, that take as parameters, depositions_points, min_pts, for the minimum point per layer to be consider not outliers, set on 50, and the layer height set as 1.

Every points are now sorted out and clean, ready to have voxel transformation.

Real-Time 3D Map of XYZ Coordinates

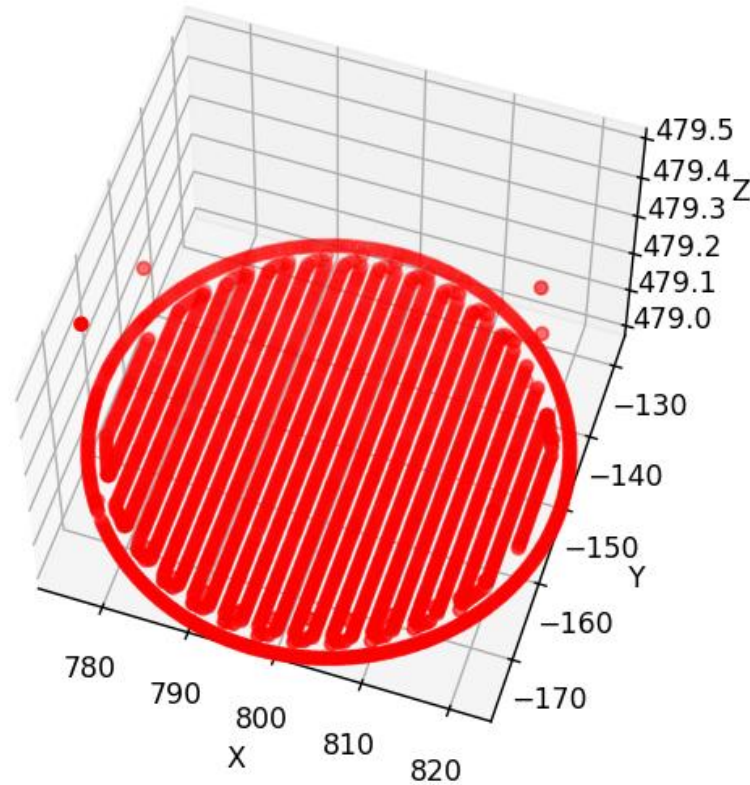


Figure 3 Coordinate fetched from Robot Studio Represented in 3D

4.3 Voxel-Based Geometry Pipeline

To create the voxel representation after filtering the outliers and cleaning the data, multiple steps are to be taken. The first is to map the coordinate to the voxel indices, then to fill gaps between indices to be sure to have the full pieces. This is connected to the aim of creating a fully accurate representation of the pieces in 3D.

4.3.1 Voxel Grid Parameters (resolution, layer thickness)

3 value are required to create the voxel representation: the number of layers, obtain once again with the function “`fetch_number_of_layer(url_number_of_layers)`” that take into parameter the URL to have access to the value of the number of layers of this pieces, it is `nz`, and will define the number of layer the voxel will have. `Ny` and `nx` will define the horizontal dimension and are defined by hand to 400 by 400 for the majority of the work, but as much as 2000 by 2000 voxel dimensions were used.

4.3.2 Deposition-Point Voxel-Index Mapping

The deposition points are then mapped to the voxel’s indices using the function “`point_to_voxel_indices()`” that takes as argument the coordinate `x`, `y`, `z` from deposition points, `nz`, `nx`, `ny`, the layer height, as well as the range of the coordinate and its minimum.

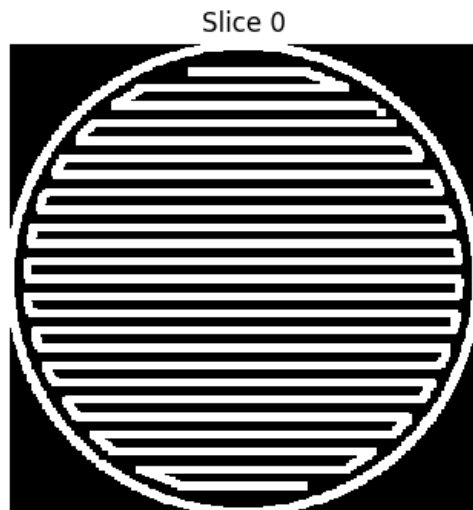
4.3.3 In-Plane Filling & Vertical Smoothing

To fill the gap between points, a fill radius of dimension 3 by 3 is used, which fills the voxel around one active voxel with a matrix 3 by 3.

Also, if needed, a vertical smoothing is applied to allow a better consistency between layers. None is done here.

4.3.4 Bounding-Box Extraction and Storage

After having a correct representation, it is saved using the “`save_bounding_boxes_from_grid()`” function, which saves the bounding boxes within every active



pixel of the piece, so it saves space. To use it later, what to do is just to reproduce a black out voxel within the bounding box dimension and put an active pixel as an active voxel. It is saved under a gzipped json to save even more space. All of that allowed us to have even more precision, to go to thousands of voxels in length and width without having to save too much, but it will take a lot of space and computer resources to do the heat simulation.

Figure 4 Voxel representation of the first layer of piece 1

4.4 Transient Heat-Transfer Simulation

The Heat-Transfer simulation is done using the gzipped json previously created. After the voxels are reproduced inside the “heat_simulation()” function using the active pixel and bounding box dimensions. The geometry of each piece is found and saved in a dictionary using the function “analyze_geometry()”, which takes as parameters active pixels and the bounding box dimensions. It then returns the density, the average thickness of the “wall” inside the piece, and the average distance within the piece.

4.4.1 ODE Solver Implementation for Each Voxel

For each voxel, this ODE equation is used to calculate and update its temperature:

$$T[z, y, x] = T + dt * \alpha * Q - \beta * (T - T_{amb}) - \gamma * (T^4 - T_{amb}^4).$$

With every piece’s part that is connected compute its own temperature according to this equation, dt is set to 1, otherwise it takes too much computer power, but for more accurate heat propagation other value must be attributed, same for steps_per_layer, which is how many seconds in the future the heat is to be predicted, it is also set to one for the same reason.

4.4.2 Empirical Parameter Calibration (α , β , γ)

The heat equation, used to calculate the temperature of each part of the pieces, is modified according to the pieces’ geometric property disclosed previously with this ratio:

$$\alpha = (0.8 * compactness + 0.75 * (1 / (1 + thickness)) + 1 * density) * ((number_layer + 11) / 13),$$

$$\beta = (0.03 + 0.01 * (1 - compactness) + 0.02 * (gap / 50.0)) * (((time_cooling))^{**2}),$$

$$\gamma = (1e-6 + 5e-6 * compactness + 1e-6 * (distance / 15.0)) * (((time_cooling))^{**1.05}).$$

The ratio within α , β , γ changed during the thesis work to try and manage to fit appropriately the reality of temperature, especially with the cooling time, which here is boosted to accelerate the learning of the AI, but should not be if more realistic over time cooling is desired. Otherwise, the equations are balanced, so the more layers, the more heat, and the slower it is to dissipate the heat. It is also heavily influenced by the compactness and density, and the heat dissipation is influenced by the gap within the piece as well as its compactness and its cooling time.

4.4.3 In-Slice Neighbourhood Averaging & Time-Stepping

Finally, the temperature of each voxel to be put in the ODE equation is calculated using the average temperature of its neighbours, using a 3*3 matrix around the voxel with a 40% ratio and 60% ratio for the voxel’s temperature in itself, it is the T in the formula.

4.4.4 Saving the heat map

Using the function “compute_piece_avg_temp()”, return the heat map as well as the average temperature of the piece to create the threshold needed later. Implemented in another function, called “save_heat_stats()”, the map is saved under a .npy file, as well as the average temperature, the file_name, the cool time, and the dimension of the map under a dictionary to access it at any time, and display all those statistics needed.

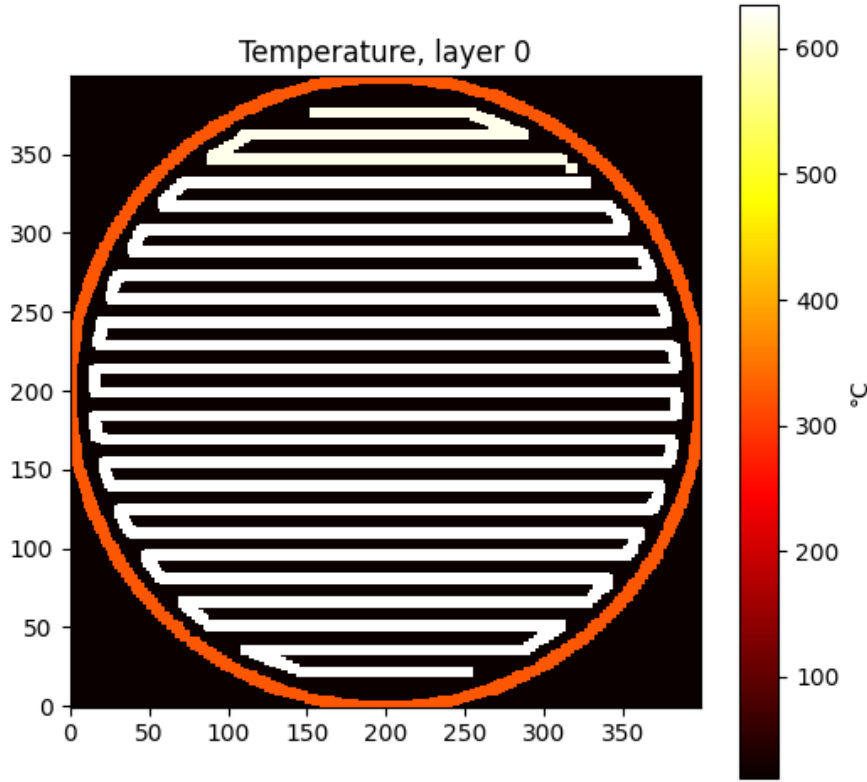


Figure 5 Heat map of piece 1

With that, the whole heat pipeline is done, and the heat representation is clear and working.

4.5 Reinforcement-Learning Driven Build-Sequence Scheduling

The build sequence scheduling is an important part of the work, as it chooses which piece to print next and touches the core of the thesis's goal, which is to save time and have good quality over the pieces printed. For that, a Reinforcement Learning model is built.

4.5.1 Design of the Reward Function

The reward function was designed so that the model gets a positive reward if it saves time. It is designed like that:

- Always -1 as a baseline penalty to discourage unnecessary delay
- +5 if it prints immediately
- + 1 if it prints within 30 seconds of waiting maximum
- -5 if there was more than 30 seconds wait

This scheme strongly rewards immediate prints, to have as little time as possible to wait, while still allowing some short waiting time.

4.5.2 Scheduling Algorithm Logic

State Vector: To be the most efficient possible, but still keeping a relatively simple model, the agent has access to the cool time of each piece, its number of layers, as well as its average temperature. The average temperature and cool time are scaled down by dividing them by 10 each.

Valid Action: The model has to choose a piece to print where its average temperature falls below a threshold of 400-degre °C. Otherwise, it waits 10 seconds before it computes again the temperature and can make a choice again, doing so until at least one piece becomes printable.

Action Selection: ϵ -greedy policy with ϵ starting at 0.2 and decaying by a factor of 0.995 per decision to a minimum of 0.05.

4.5.3 Q-Learning Algorithm and Hyperparameters

The Q-learning update: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \cdot a' \max_{a'} Q(s', a') - Q(s, a)]$. was implemented with a learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.9$, as specified by the Q-Agent constructor in `q_agent.py`. These values were selected based on an initial set of short trial episodes, which indicated stable convergence behaviour without further tuning.

4.5.4 Integration with Thermal Model and Python Pipeline

To integrate the Reinforcement Learning model with the Python pipeline and the thermal model, which is done inside the pipeline, the model, a Q-table, is first loaded if it exists; otherwise, it is initialized. After printing one layer per piece, and doing a heat simulation in all of them in a dictionary called “stats” using “`save_heat_stats()`”. A new loop is started where the model observes the new state, computes the reward for the preceding action. Then the model takes an action using “`agent.choose_action(state, valid_actions)`” and choose an available piece to be printed, the rewards is then calculated, the pieces are printed using “`set_piece_choice(choice)`” with the choice of the model. After the layer is complete “`agent.update(prev_state, prev_action, reward, state, valid_actions)`” is called and ϵ is decayed via “`agent.decay_epsilon()`”.

Finally, when all the pieces are printed, the new agent is saved, and a graph of the rewards progress is displayed. And the total time to print everything is shown.

4.6 Experimental Trials and Validation

To train and optimize the build sequence, a series of trials were done using different heat simulation parameters, blank RL models, using different numbers of layers to print, and different path choosing algorithms to compare.

4.6.1 Selection of Test Geometries and Simulation Parameters

Experience included different piece complexities, layer numbers ranging from 5 to 10, and various empirical parameter calibrations to explore their impact on the build sequence optimization. For instance:

Test Case 1 (Trial 1 - Figure 6 & 7):

Accelerated cooling parameters, blank RL model:

- α : $(0.8 \times \text{compactness} + 0.75 \times (1 / (1 + \text{thickness})) + 1 \times \text{density}) \times ((\text{number_layer} + 11) / 13)$
- β : $(0.03 + 0.01 \times (1 - \text{compactness}) + 0.02 \times (\text{gap} / 50.0)) \times (\text{cool_time}^2)$
- γ : $(1e-6 + 5e-6 \times \text{compactness} + 1e-6 \times (\text{distance} / 15.0)) \times (\text{cool_time}^{1.05})$

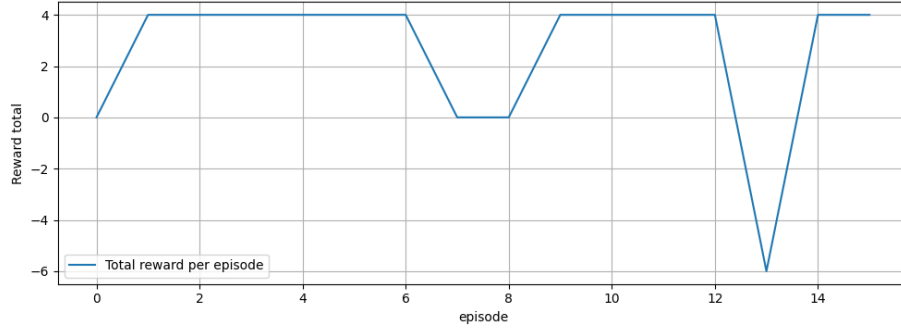


Figure 6 Evolution of AI performance for the first run under case 1 condition, reward per loop according to which loop run

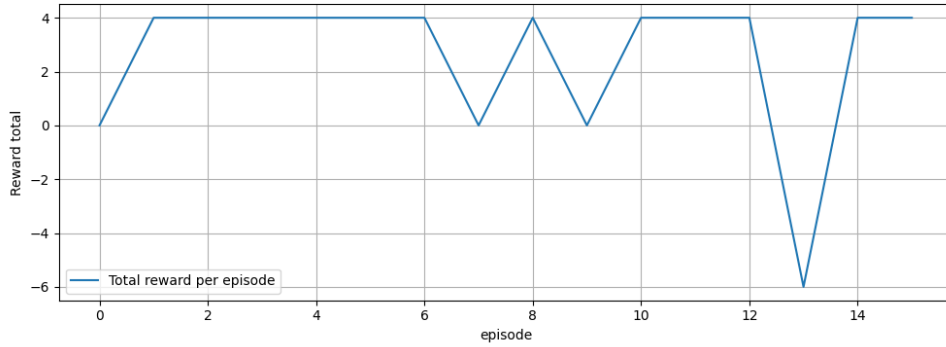


Figure 7 Evolution of AI performance for the second run under case 1 condition, reward per loop according to which loop run

Result:

- RL: 1548.43 s
- Algorithm: 1604.12 s

Test Case 2 (Trial 2 - Figure 8):

Modified parameters for extended cooling and sensitivity exploration, with the model that trained under case 1:

- α : $(0.4 \times \text{compactness} + 0.5 \times (1 / (1 + \text{thickness})) + 0.9 \times \text{density}) \times ((\text{number_layer} + 8) / 10)$
- β : $(0.02 + 0.03 \times (1 - \text{compactness}) + 0.01 \times (\text{gap} / 50.0)) \times ((\text{cool_time} + 0.01)^3)$

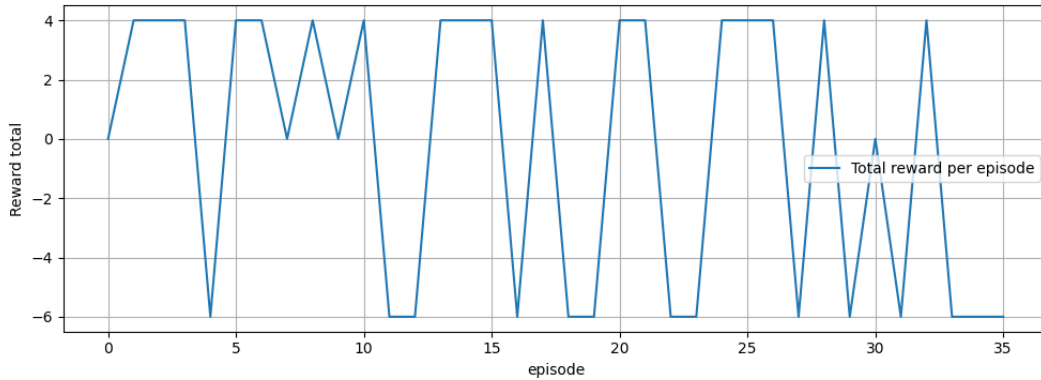


Figure 8 Evolution of AI performance for the first run under the case 2 condition, reward per loop according to which loop run

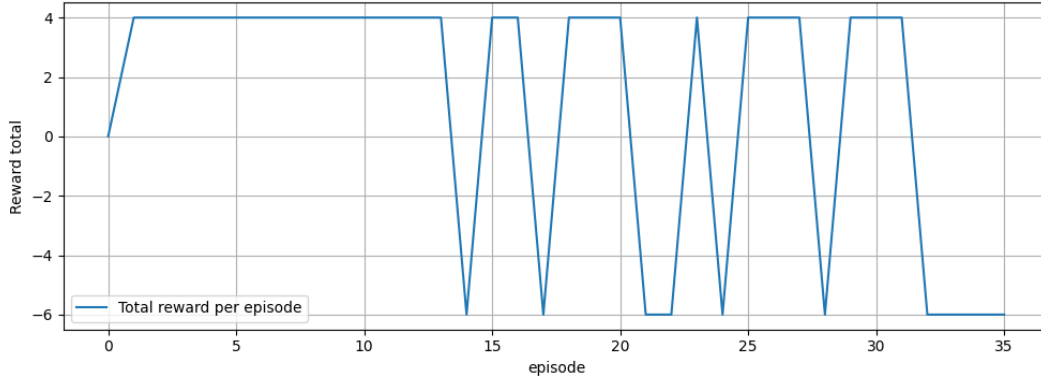


Figure 9 Evolution of AI performance for the first run under the case 3 condition, reward per loop according to which loop run with threshold of 400 degree

- $\gamma: (1e-6 + 5e-6 \times \text{compactness} + 1e-6 \times (\text{distance} / 10.0)) \times ((\text{cool_time} + 1/120)^{1.05})$

Result:

- RL: 7274.27 s
- Algorithm: 7247.04 s

Test Case 3 (Trial 3 – Figure 9):

Adjusted parameters for increased realism, with the model now trained on the first two cases:

- $\alpha: (0.8 \times \text{compactness} + 0.9 \times (1 / (1 + \text{thickness})) + 0.8 \times \text{density}) \times ((\text{number_layer} + 18) / 20)$
- $\beta: (0.03 + 0.01 \times (1 - \text{compactness}) + 0.05 \times (\text{gap} / 50.0)) \times (\text{cool_time}^2)$
(cool_time / 10 for realism)
- $\gamma: (1e-6 + 5e-6 \times \text{compactness} + 1e-6 \times (\text{distance} / 25.0)) \times (\text{cool_time}^{1.03})$
(cool_time / 12 for realism)

Result:

- RL: 6314.76 s
- Algorithm: 7271.21 s (RL significantly better)

Test Case 4 (Trial 4 – Figure 10):

Further adjusted parameters for fine-tuning realism and balance, started with a blank RL model:

- $\alpha: (0.8 \times \text{compactness} + 0.9 \times (1 / (1 + \text{thickness})) + 0.8 \times \text{density}) \times ((\text{number_layer} + 23) / 25)$
- $\beta: (0.03 + 0.01 \times (1 - \text{compactness}) + 0.05 \times (\text{gap} / 50.0)) \times (\text{cool_time}^2)$
(cool_time / 10 for realism)

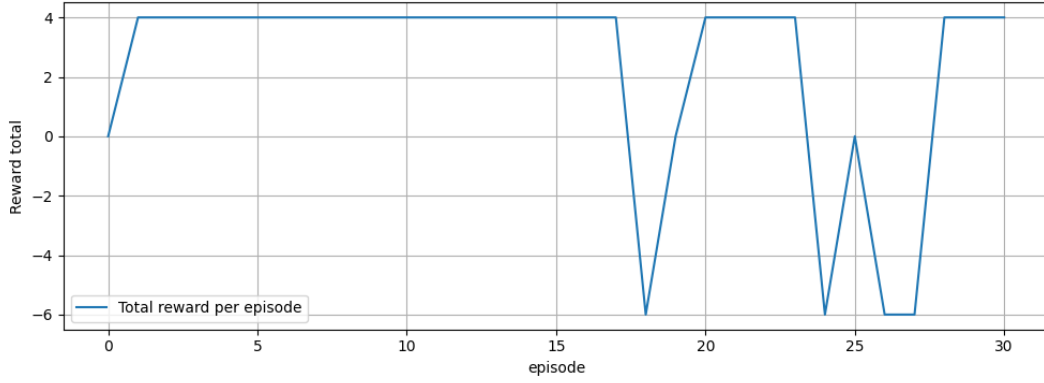


Figure 10 Evolution of AI performance for the first run under the case 4 condition, reward per loop according to which loop run

- $\gamma: (1e-6 + 5e-6 \times \text{compactness} + 1e-6 \times (\text{distance} / 50.0)) \times (\text{cool_time}^{1.05})$
(cool_time / 12 for realism)

Results (First Run):

- RL: 2668.89 s
- Algorithm: 2629.95 s (*Algorithm slightly better initially*)

Test Case 5 (Trial 5 - Figure 11):

Identical parameters to Test Case 4, but with an additional run to confirm RL performance improvement after continued training.

Results (Second Run after additional training):

- RL: 2575.98 s (*Improved from previous RL run*)
- Algorithm: 2629.95 s (*Algorithm unchanged, RL now better*)

All the trials were done, and only the most relevant ones were retained.

4.6.2 Trial Protocol (number of runs, data captured)

A systematic experimental protocol was followed:

- Number of trials: Over 100 experiments.
- Data captured: TCP coordinates, voxel representations, cooling times, temperature distributions, RL rewards, and total build times.

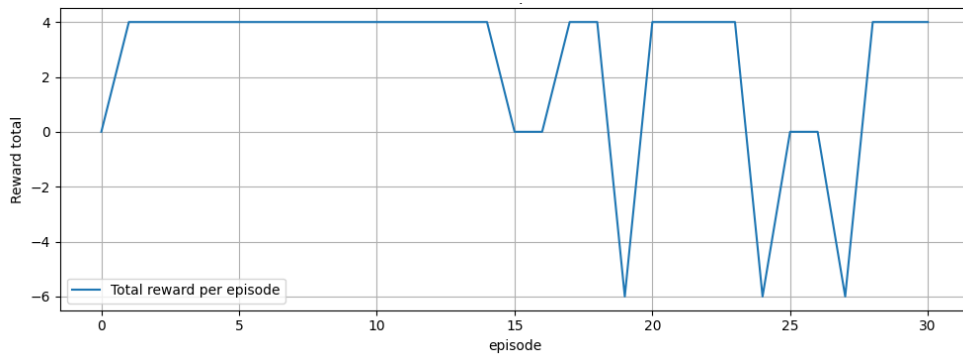


Figure 11 Evolution of AI performance for the second run under the case 4 condition, reward per loop according to which loop run

Only the relevant experiments were selected and are displayed here. Most of them had to be aborted during the training to change values to optimize the heat or the Reinforcement Learning model.

4.6.3 Performance Metrics and build time saving

Performance was primarily evaluated using total build time, reinforcement learning reward metrics, and comparative performance against a deterministic selection algorithm.

Interpretation:

RL consistently improved total printing time in scenarios tuned for realistic and computationally balanced parameters.

RL exhibited slight disadvantages in some specific setups, indicating sensitivity to parameter tuning.

The RL model constantly receives more rewards and fewer penalties, indicating that it has learned and improved.

4.6.4 Error Sources and Uncertainty Quantification

Several uncertainty factors and potential sources of error were identified:

- **Simulation** vs. **Reality** **Discrepancies:**
Thermal property assumptions are correct, but not sticking to reality perfectly, certain uncertainties are still present, but it is correct and sufficient for the work.
- **Voxel** **Resolution** **Limitations:**
Small geometric features below ~2mm might be neglected due to voxel resolution limits, causing slight deviations. To fix it, putting the voxel dimension higher would work, but it would take more computing power.
- **Cooling** **Time** **Measurement:**
Timing accuracy (using Python's performance counter) was considered highly reliable, thus negligible error contribution.
- **RL** **Model** **Variability:**
The stochastic nature of RL training and ϵ -greedy policy may introduce variability. This was overcome by averaging results over multiple trials.

5 Results and discussion

The results presented in this thesis illustrate the potential of combining both voxel-based heat simulation and Reinforcement learning (RL) to improve efficiency in the metal 3D printing process, especially through Wire Arc Additive Manufacturing (WAAM). Multiple trials were conducted, and the findings are summarized and discussed below.

5.1 Summary of Key Findings

Conducted experiment shows that RL path selection performs better than a deterministic algorithm and outperforms a simple one that just prints the pieces in their order, particularly when the heat simulation parameters were tuned to realistically simulate a cooling time scenario. This goes accordingly to the initial expectation based on the literature studies, which sheds light on the adaptability and optimization of RL in a dynamically changing environment and on path selection.

5.2 Detailed Analysis of Experimental Results

Table 1: Trial Results

Test Case	RL Total Time (s)	Algorithm Total Time (s)	Time Improvement (%)
5 layers (Case 1)	1548.43	1604.12	+3.47% (RL Faster)
5 layers (additional run)	1560.87	1604.12	+2.70% (RL Faster)
10 layers (Case 2)	7274.27	7247.04	-0.38% (Algorithm Faster)
10 layers Realism Adjusted (Case 3)	6314.76	7271.21	+13.16% (RL Faster)
10 layers and 5 for one piece (Case 4 Second Trial)	2575.98	2629.95	+2.05% (RL Faster)

As seen in Table 1, **test case 1** (Baseline Cooling Acceleration) demonstrates initial success, with a better time from the RL model than the deterministic algorithm by about 3.47%, reducing total printing time from 1604.12 s to 1548.43 s. This early success validates the choice and feasibility of using an RL model for choosing a path.

The other test helps reinforce this idea, even without always an improvement.

In **test case 2** (Extended Cooling and Parameter Sensitivity) revealed a more nuanced outcome. The deterministic algorithm outperformed the RL model (with RL: 7274.27 s vs. Algorithm: 7247.04 s), which highlighted sensitivity to specific parameter settings.

In contrast, **Test Case 3** (Realism Adjusted Parameters) yielded significant results and improvement, with RL reducing total build time by approximately 13.16 % compared to deterministic scheduling (RL: 6314.76 s vs. Algorithm: 7271.21 s). This demonstrated RL's

adaptability in realistically tuned thermal simulations, emphasizing its capability in practical, real-world scenarios.

Test Cases 4 (Further Realism and Training Improvements) further reinforced the effectiveness of RL when parameters were consistently realistic. Initially, RL slightly underperformed in Test Case 4 (RL: 2668.89 s, Algorithm: 2629.95 s), yet with continued training and reapplication, RL surpassed the deterministic algorithm, achieving a reduction in build time from 2629.95 s (Algorithm) to 2575.98 s (RL). This confirms that with adequate training and realistic parameter tuning, RL's adaptive advantages become pronounced.

5.3 Reinforcement Learning Model Performance

Analysis of the RL reward progression across the trial, figures 6 to 11, the RL is getting more and more rewards over episodes, signifying successful learning over multiple loops. The early fluctuation observed in Figures 6 to 8 was typical for the early stages of development. In the following trials, as seen in figures 9 to 11, stable reward improvement is evident. Demonstrating the model's capability to learn and optimize progressively.

6 Conclusion

The primary aim of this thesis was to enhance efficiency in metal 3D printing through the integration of advanced heat simulation and Reinforcement Learning algorithms. The objective has been successfully met with the development of a comprehensive, voxel-based simulation environment coupled with an RL-driven path model for Wire Additive Arc Manufacturing.

Achievement of Objectives:

The objectives initially stated in this thesis were clearly addressed:

- **Physics-based Model Development and Validation:**
A 3D printing simulation was developed to accurately simulate the real machine and allowed accurate representation and heat model of the pieces printed within this simulation.
- **Accurate Representation of Printed Pieces:**
The voxel-based approach allowed a high-fidelity geometric representation used for accurate thermal representation. The developed pipeline consistently generates precise, computationally efficient geometries suitable for layer-by-layer heat analysis.
- **Thermal Simulation of Printed Pieces:**
Thermal simulation was performed effectively, with some simplifications, results have matched enough realistic thermal behaviours for the thesis needs, validating the accuracy and practicality of the heat model. The empirical parameters captured the thermal dynamics associated with geometry and cooling times.
- **RL-based Optimization of the Build Sequence:**
The Reinforcement Learning model effectively optimized the build sequence, reducing total print time and improving piece quality. These findings support the hypothesis that adaptive learning methods outperformed the conventional approach and the Candide algorithm in dynamically varying thermal conditions.

The validity of these results and conclusions are based on numerous trials and comparative performance assessment. However, the limitations due to computational constraints, lack of real data, and direct validation from a physical model should be acknowledged. These constraints induce caution regarding direct generalizability to industrial applications without further testing.

6.1 Future Work and Research

While the current results are promising, several ways of improving the work and results exist:

- **Real-world validation:**
Future research could focus on real-world testing and validation of the developed model on actual WAAM hardware, addressing any differences between the simulation and the real-world machine.
- **Advanced RL methods:**
The RL model developed during this thesis is a simple one; a more advanced model, such as a Deep Q-network, taking into parameters the heat map instead of just the temperature, could be a huge step in further refining the performance.

- **Enhanced Thermal Modeling:**
The thermal model is sufficient to train the RL model, but a lot of improvement can be done, such as having a cool time for each voxel instead of pieces, enhancing the thermal behaviour and parameters using ML and a real heat map of metal 3D printing pieces.
- **Scalability and Complexity:**
Further studies could explore the application of the developed methods to larger and more complex geometries and production environments to assess scalability and industrial capabilities.

6.2 Critical Discussion

Throughout this project, several insights emerged, such as:

Doing a heat simulation and voxel representation is hard, but it is rewarding regarding the efficiency of the work done. Balancing the heat equation, to get a correct representation that can be used for making decisions and representing the cool time of each piece, was hard and took a long time, lots of trials to refine and find a correct one that can be used.

The RL model, sensitivity to the reward system and parameters, has shown itself to be important to have correct results. A trial-and-error approach was necessary to calibrate it correctly.

The iterative and experimental approach provided personal experiences, shows how the methods have to be very rigorous, a methodological approach, and that there is always room to have improvement, and the experiment can always be better.

6.3 Generalization of the result

The results of this study can be used beyond the specific WAAM case studied here. The Voxel based heat simulation approach combined with Reinforcement Learning for dynamic scheduling can be used not only in WAAM, but also in other fields of DED where thermal management is crucial.

Additionally, the principles of dynamic decision-making developed in this study can be applied in a broader context such as robotics, automated manufacturing, or even other industrial automation fields requiring real-time optimization.

7 References

- [1] J. Jean, "GitHub pages for the thesis," [Online]. Available: https://github.com/Justine-IA/3D_printing.
- [2] Unknown, "Advanced Metal 3D Printing Center," Michigan Technological University, 2025. [Online]. Available: <https://www.mtu.edu/metal-3d/services/orientation/methods/>. [Använd 05 2025].
- [3] D. M. Z. B. V. A. B. S. B. A. S. J. L. E. E. N. Svetlizky D, "Directed energy deposition (DED) additive manufacturing: Physical characteristics, defects, challenges and applications," *Materials Today*, vol. 49, nr 10.1016/j.mattod.2021.03.020, p. 25, 30 11 2021.
- [4] Unknown, "Wire Arc Additive Manufacturing (WAAM)," Migal, 2025. [Online]. Available: [https://www.migal.co/en/waam#:~:text=Wire%20Arc%20Additive%20Manufacturing%20\(WAAM\)%20is%20a%20manufacturing%20process%20used,print%20or%20repair%20metal%20parts..](https://www.migal.co/en/waam#:~:text=Wire%20Arc%20Additive%20Manufacturing%20(WAAM)%20is%20a%20manufacturing%20process%20used,print%20or%20repair%20metal%20parts..) [Använd 05 2025].
- [5] M. S. A. S. a. U. R. O. MOKROV, "A FINE MODIFICATION OF THE DOUBLE," i *ISF – Welding and Joining Institute*, Aachen, 2019.
- [6] M. T. a. V. Petkov, "A THERMAL MODEL FOR WIRE ARC ADDITIVE MANUFACTURING," in *14th International Scientific and Practical Conference*, Latvia, 2024.
- [7] W. Shen, "Research on virtual simulation design of ABB robot welding operation based on Robotstudio," i *2020 IEEE International Conference on Artificial Intelligence and Computer Applications*, Dalian, China, 2020.
- [8] S. Y. S. Y. J. M. Ruixiang Cao, "Printing, Poxel: Voxel Reconstruction for 3D," p. 3, 2025.
- [9] A. P. Alexey Tolok, "Functional-Voxel Method in Problems of Geometric," Moscow, 2020.
- [10] F. F. O. F. M. M. Etienne Chassaing, "Thermoxels: a voxel-based method to generate simulation-ready 3D thermal models," Lausanne, 2025.
- [11] D. C. X. Z. C. G. L. L. Y. H. W. B. Xijun Zhang, "Machine learning-driven 3D printing: A review," *Applied Materials Today*, vol. 39, nr doi.org/10.1016/j.apmt.2024.102306, 2024.
- [12] V. S. N. P. A. M. M. a. M. P. F. J. M. A. Fabio Parisi, "A new concept for large additive manufacturing in construction: tower crane-based 3D printing controlled by deep reinforcement learning," *Construciton Innovation*, vol. 24, nr doi/10.1108/ci-10-2022-0278/, p. 25, 2023.
- [13] A. D. A. A. E.-K. S. A. Hassanien, "Artificial Intelligence in 3D Printing," doi.org/10.1007/978-981-33-6129-4_6, Springer, Singapore, 28 May 2021.

- [14] M. D. M. M. S. Z. D. E. Rojek I, "Optimization of Extrusion-Based 3D Printing Process Using Neural Networks for Sustainable Development," *Materials*, vol. 11, nr 2737, p. 14, 2021.